

AlgebraicPetri SIR callbacks

Lockdown and vaccination interventions in a labelled SIR model

This vignette lowers intervention programs to `DiffEqCallbacks.jl` callbacks and applies them to an SIR model implemented with `AlgebraicPetri.jl`. It uses the same parameterization as the `StockFlow` and `AlgebraicDynamics` vignettes, following [epirecipes/sir-julia](https://epirecipes.github.io/sir-julia/): $S(0)=990$, $I(0)=10$, $R(0)=0$, contact rate $c=10$, transmission probability $\beta=0.05$, and recovery rate $\gamma=0.25$.

The key point is that `AlgebraicPetri` models naturally use labels for species and transition rates. `PetriIndexing(parameters=[:inf, :rec], states=[:S, :I, :R])` therefore uses the labels themselves as selectors, avoiding brittle positional indexing.

```
using AlgebraicPetri
using CategoricalInterventions
using DiffEqCallbacks
using LabelledArrays
using OrdinaryDiffEq
using Plots

default(; linewidth=2, grid=false)

sir_model = LabelledPetriNet(
    [:S, :I, :R],
    :inf => ((:S, :I) => (:I, :I)),
    :rec => (:I => :R),
)

u0 = LVector{S=990.0, I=10.0, R=0.0}
p0 = LVector{inf=0.05 * 10.0 / sum(u0), rec=0.25}
tspan = (0.0, 40.0)

prob = ODEProblem(vectorfield(sir_model), u0, tspan, p0)
baseline = solve(prob, Tsit5(); saveat=0.5)

baseline.retcode, baseline[3, end], sum(baseline.u[end])
```

```
(SciMLBase.ReturnCode.Success, 775.6835919143083, 1000.0000000000002)
```

A parameter intervention: lockdown

A lockdown is represented as a multiplicative reduction in the infection transition rate `:inf` over an interval.

```
indexing = PetriIndexing(parameters=[:inf, :rec], states=[:S, :I, :R])

inf_rate = Target(
    :inf;
    kind=Parameter,
    value_type=Float64,
    algebra=MultiplicativeAlgebra(),
)

lockdown = InterventionProgram(
    InterventionAtom(:lockdown, inf_rate, Interval(5.0, 25.0), Scale(0.5)),
)

lockdown_cb = to_callback(lockdown, indexing; baseline_p=p0)
lockdown_sol = solve(prob, Tsit5(); callback=lockdown_cb, saveat=0.5)

lockdown_sol.retcode, lockdown_sol[3, end], sum(lockdown_sol.u[end])
```

```
(SciMLBase.ReturnCode.Success, 354.0601008712396, 1000.0000000000002)
```

The lower final recovered count indicates that the reduced infection rate prevented some infections.

A state intervention: vaccination

Vaccination is a state transfer rather than a parameter change. It can be encoded as two simultaneous state pulses: remove people from `:S` and add them to `:R`.

```
susceptible = Target(:S; kind=State, value_type=Float64, algebra=AdditiveAlgebra())
recovered = Target(:R; kind=State, value_type=Float64, algebra=AdditiveAlgebra())

vaccination = InterventionProgram(
    InterventionAtom(:vaccinate_from_s, susceptible, Interval(15.0, 16.0), Add(-50.0)),
```

```

    InterventionAtom(:vaccinate_to_r, recovered, Interval(15.0, 16.0), Add(50.0)),
)

vaccination_cb = to_callback(vaccination, indexing)
vaccination_sol = solve(prob, Tsit5(); callback=vaccination_cb, saveat=0.5)

vaccination_sol.retcode, vaccination_sol[1, end], sum(vaccination_sol.u[end])

```

```
(SciMLBase.ReturnCode.Success, 213.66091888858364, 999.9999999999999)
```

The vaccination callback preserves total population because the state intervention is encoded as a transfer.

Composing lockdown and vaccination

The parameter intervention and the state transfer target different components, so they compose without conflict.

```

combined = compose_interventions(lockdown, vaccination)
combined_cb = to_callback(combined, indexing; baseline_p=p0)
combined_sol = solve(prob, Tsit5(); callback=combined_cb, saveat=0.5)

(
    baseline_R = baseline[3, end],
    lockdown_R = lockdown_sol[3, end],
    vaccination_S = vaccination_sol[1, end],
    combined_R = combined_sol[3, end],
    combined_total = sum(combined_sol.u[end]),
)

```

```
(baseline_R = 775.6835919143083, lockdown_R = 354.0601008712396, vaccination_S = 213.66091888858364)
```

The same intervention program can be inspected before it is lowered to callbacks.

```
[(epoch.support, collect(keys(epoch.effects))) for epoch in epochize(combined)]
```

```

3-element Vector{Tuple{Interval{Float64}, Vector{Symbol}}}:
 ([5.0, 15.0), [:inf])
 ([15.0, 16.0), [:R, :inf, :S])
 ([16.0, 25.0), [:inf])

```

Cross-check against the same SIR equations

The same intervention program should give the same trajectory when applied to the direct SIR equations. This guards against accidentally changing the meaning of :inf, :rec, :S, :I, or :R across representations.

```
function sir_ode(u, p, t)
    infection = p.inf * u.S * u.I
    recovery = p.rec * u.I
    return LVector(S=-infection, I=infection - recovery, R=recovery)
end

reference_prob = ODEProblem(sir_ode, u0, tspan, p0)

reference_baseline = solve(reference_prob, Tsit5(); saveat=0.5)
reference_combined = solve(
    reference_prob,
    Tsit5();
    callback=to_callback(combined, indexing; baseline_p=p0),
    saveat=0.5,
)

max_reference_difference = maximum(abs, Array(baseline) .- Array(reference_baseline))
max_combined_difference = maximum(abs, Array(combined_sol) .- Array(reference_combined))

max_reference_difference, max_combined_difference
```

```
(1.755369112288463e-7, 1.8250489119964186e-8)
```

```
max_reference_difference < 1e-6 && max_combined_difference < 1e-6
```

```
true
```

Plotting the intervention scenarios

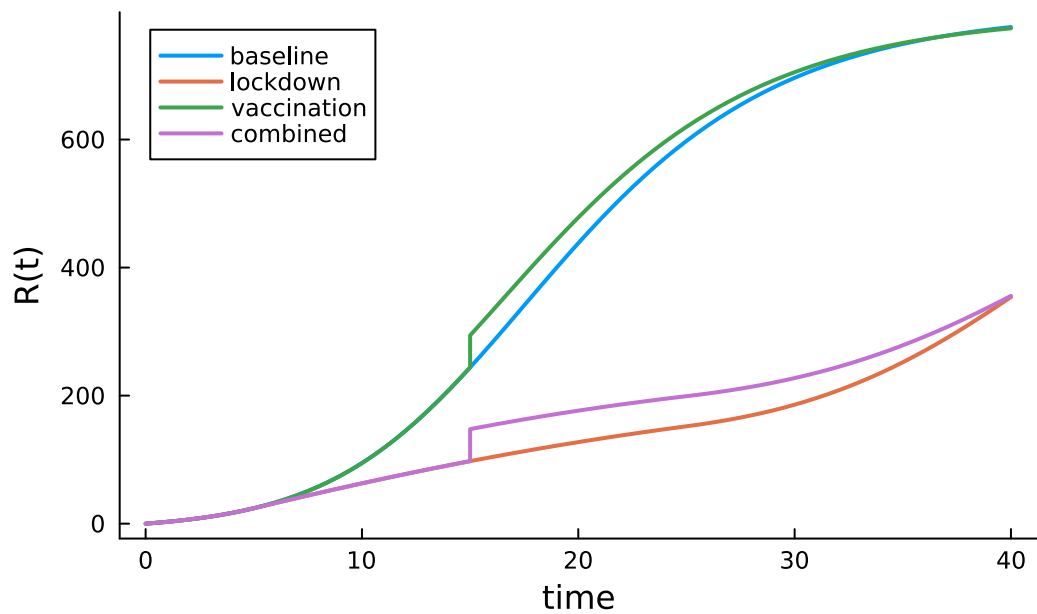
The final recovered counts summarize the epidemic size, but plotting the whole trajectory shows when each intervention has its main effect.

```

plot(
    baseline.t,
    baseline[3, :];
    label="baseline",
    xlabel="time",
    ylabel="R(t)",
    title="Recovered population under intervention scenarios",
)
plot!(lockdown_sol.t, lockdown_sol[3, :]; label="lockdown")
plot!(vaccination_sol.t, vaccination_sol[3, :]; label="vaccination")
plot!(combined_sol.t, combined_sol[3, :]; label="combined")

```

Recovered population under intervention scenari



The combined scenario can also be inspected by compartment.

```

plot(
    combined_sol.t,
    combined_sol[1, :];
    label="S",
    xlabel="time",
    ylabel="population",
    title="Combined lockdown and vaccination scenario",
)

```

```
plot!(combined_sol.t, combined_sol[2, :]; label="I")  
plot!(combined_sol.t, combined_sol[3, :]; label="R")
```

